

LLM과 Chatbot을 결합한 업무 Q&A봇 구현 및 설계

Implementation and Architecture Design of a business Q&A bot Combining LLM and Chatbot

최 남 규* Choi, Nam Kyu
이 혜 은** Lee, Hye Eun
전 병 욱** Jun, Byeong Wook
박 재 일*** Park, Jae Il

Abstract

Purpose Our goal is to propose an efficient and scalable Q&A bot platform. This platform is designed to enhance LLM performance and streamline chatbot construction by integrating various high-quality libraries and development tools. Additionally, we aim to create an application that allows for immediate correction of chatbot responses if they are incorrect.

Methods We apply concepts such as Multi LLM, multi RAG, and multi embedding. Additionally, we integrate a similarity-based Chatbot by invoking the similarity document retrieval module in the RAG-based LLM platform.

Results We enabled the RAG-based LLM to generate responses by first uploading documents without scenario registration. Additionally, to address cases where LLM couldn't respond, we registered query-response sets in the intent, facilitating easy correction of Chatbot responses.

Conclusion We propose a flexible Chatbot architecture. Current research lacks prompt responses to LLM errors, and traditional Chatbots need retraining for intent changes. We suggest integrating a learning-free Chatbot for instant error correction during chats, with reporting and rectifying measures.

Keywords : *LLM, RAG, Chatbot, Q&A bot, Fine-tuning*

* 주저자, (주)이아이씨엔 부장, E-mail: hans@eicn.co.kr

** 공동저자, 아주대학교 산업공학과 박사과정수료, E-mail: vrmania@ikw.ac.kr

*** 공동저자, (주)이아이씨엔 이사, E-mail: bwjun@eicn.co.kr

**** 교신저자, 아주대학교 산업공학과 교수, E-mail: jipark@ajou.ac.kr

† 2024년도 산업통상자원부 및 한국산업기술기술평가원(KEIT) 연구비 지원에 의한 연구(‘과제번호 20023916’)

† 2024년도 과학기술정보통신부 및 정보통신산업진흥원 ICT기금사업 AI바우처관련 논문(‘접수번호 R-20240320-022640’)

투고일 2024.06.10

심사일 2024.06.12

게재일 2024.06.30

1. 서론

대형언어모델(Large Language Models) 기반의 Chatbot은 다양한 표현을 활용하여 어휘맥락에 부합하는 답변을 제공하는 장점을 갖고 있어 전통적인 Chatbot 기술을 대체하는 기술로 주목받고 있다[1]. 그러나 업무 Chatbot은 명확하고 정확한 정보를 전달하는 것이 중요하며, 업무 수행에 있어 혼란을 초래할 만한 잘못된 정보를 제공하는 경우 답변의 신뢰성이 저하되므로, 보다 통제가 용이한 형태의 Chatbot이 선호된다[2].

LLM은 도메인의 경계없이 방대한 범위의 정보를 학습하기 때문에 엉뚱한 답변을 하거나 학습 이전의 지식에 대해 답변하지 못하는 등의 문제가 발생할 수 있다[3]. 이러한 문제점을 해결하기 위해 모델 외부에서 데이터를 검색하고, 이를 LLM에 전달하여 응답을 생성하는 검색증강생성(Retrieval Augmented Generation, 이하 RAG)방식에 대한 연구가 활발하게 진행되고 있다[4]. RAG 기반 LLM은 문서 검색과 생성을 결합하여 사용자의 질문에 대한 답변을 제공하는 방식으로, 도메인에 적합한 정보 제공을 가능하게 한다. 하지만 RAG 기반 LLM의 성능은 등록하는 원본 문서의 구조나 형태에 많은 영향을 받으며, 특히 불특정 다수가 사용하는 SaaS형태의 서비스에서는 잘못된 답변을 보정하기가 쉽지 않다 [4][5].

이에 따라, 본 연구에서는 원인 파악에 도움이 되는 방법을 제시하고, 유사도 기반 Chatbot 모델을 활용하여 잘못된 답변을 보정하는 방법을 제안한다. 또한 사용자나 관리자가 답변에 대한 오류를 발견했을 때 오류를 신고하고, 담당자가 조치할 수 있는 프로세스를 제안한다. 또한, 최신 기술이 접목된 RAG 기반 Chatbot시스템의 특성상 기술변화와 요구사항 변경이 빈번하므로 접목해야 할 기술 스택 변경이 용이해야 하므로 본 연

구에서는 유연하고 확장할 수 있는 아키텍처를 기반으로 설계 및 구현하였다. 본 연구에서는 이러한 제안을 소개하기 전에 관련 연구와 기존 연구의 한계를 논의한다.

2. 선행연구

2.1 Chatbot

Chatbot은 대화창에 문자나 음성을 입력하여 대화를 통해 정보를 제공하거나 처리하는 프로그램이다. 이용자의 의도를 정확히 파악하여 적합한 답변을 하거나 업무 처리를 하는 것이 중요하다. 사용자의 의도를 ‘인텐트(Intent)’라고 하며, 인텐트는 복합명사 또는 동명사 형태로 간결하게 입력할 수 있으며, 고객이 질문할 가능성이 있는 여러 질문과 그에 사용하는 답변 쌍을 설정하여 사용한다. 인텐트에서는 해당 도메인에 관련된 단어들이 사용되며, 이를 ‘개체’ 또는 ‘엔티티(Entity)’라고 한다. 엔티티는 인텐트 내의 질문이나 답변에 변수로 설정되어 질문의 수를 줄일 수도 있고, 인텐트를 식별하는 용도나 계층구조의 인텐트인 경우 다음 단계로 넘어가는 매개변수로 사용될 수도 있지만, 본 연구에서는 엔티티의 매개변수로의 활용은 다루지 않는다.

Chatbot은 규칙 기반 Chatbot, 검색 기반 Chatbot, 코퍼스 기반 Chatbot, LLM 기반 Chatbot으로 구분할 수 있다[2][6]. 규칙 기반 Chatbot은 사전 정의된 규칙 및 응답 집합에 따라 작동하고 메뉴방식의 Chatbot에 적합하며 자연어 기반의 처리가 어려운 문제가 있다. 검색 기반 Chatbot은 인텐트별로 질문과 응답 또는 로직을 등록하고 질문이 입력되었을 때 유사한 내용을 검색하여 해당 인텐트의 응답 또는 로직을 수행한다.

코퍼스 기반 Chatbot은 코퍼스 기반으로 인텐트와 엔티티를 학습하고 추론하는 방식으로 작동한다. 태스크 지향 Chatbot의 경우 BERT와 같은 구분모델을 활용하여 인텐트를 분류하는 경우가 많으며, 엔티티를 식별하기 위해 개체명 인식(Named Entity Recognition) 기술이 활용된다. 코퍼스 기반 Chatbot을 학습하는데 상당한 비용과 시간이 들며, 고객이 질문할 때 인텐트를 식별하지 못했을 경우 인텐트를 수정한 후 재학습하고 해당 인텐트의 식별이 되는지 다시 테스트하는 과정을 거쳐야 한다[6][7][8]. 이러한 이유로 기존연구에서는 즉각적인 대응이 어려울 수 있다. 또한, 학습할 때도 충분한 학습데이터가 필요하며, 인텐트 분류가 모호한 경우 분류모델의 신뢰도 임계값을 정하는데 어려움이 있다[9], 태스크 지향 Chatbot의 경우 전문 용어들이 많이 사용되는데 고유명사에 비해 전문 용어에 대한 개체명 인식 오류율이 높다는 문제가 있다[10].

최근에는 LLM 기반 Chatbot이 RAG 기술과 결합하여 환각 증상을 개선하기 위해 모델 외부에서 데이터를 검색하고, 이러한 검색 결과와 질문을 LLM이 받아서 답변을 생성하는 RAG방식에 대한 연구가 다수 진행되고 있다[3][4][5][11][12].

2.2 기술스택

텍스트 마이닝 등을 활용한 전통적인 자연어 처리 솔루션은 Java로 부분적으로 개발될 수 있었지만, 최근 배포되는 LLM 오케스트레이션 프레임워크나 GPU 기반의 LLM 서버 구현은 대부분 Python 환경에서 진행된다. LLM과 Chatbot을 결합한 업무 Q&A봇 구현에 필요한 중요한 기술 스택은 다음과 같다. LLM 서버, LLM 오케스트레이션 프레임워크, 임베딩 모델, RAG를 위한 벡터데이터베이스 처리 기술, 웹 프레임워크, 멀티프로세스 기술, API호출 제한 기술, 데이터베이스 처리 프레임워크 등이 있다.

2.2.1 LLM 서버 및 선정

LLM 기반 서비스를 개발할 때는 각 상황에 따라 클라우드 기반 LLM의 API를 이용하거나, 오픈소스 기반 LLM을 설치하여 서비스를 할 수 있다. 모델을 개발한 조직이 지속적인 지원을 할 수 있는 곳인지 점검이 필요하며, 출시된 지 얼마 안 된 모델보다는 커뮤니티에 의해 많이 사용되고 검증된 모델을 도입하는 것이 좋다. 클라우드



Fig 1. LLM 비교: Quality, Speed, Price [13] , 2024.4.28기준

기반 LLM의 API를 이용할 때에는 비용, 속도, 성능을 고려하여 선정한다. 오픈소스 기반 LLM의 경우 한국어처리를 위한 모델 Alignment가 필요한지 점검하는 것과 예산범위 내의 인프라에서 동작할 수 있는 모델인지 파라미터 수를 통해 가늠하며, 양자화/경량화 적용이 가능한지 고려하는 것이 중요하다. 특히 추론과 학습 시 필요한 GPU의 성능이 다르므로 상황에 따라 확인이 필요하다. LLM 비교[13]와 같은 벤치마크 사이트를 활용하여 비용 대비 성능에 대한 가성비를 판단할 수 있고, 이 벤치마크의 결과로 확인했을 때 openAI의 GPT 3.5 Turbo, 구글의 Gemini 1.0 Pro, Claude 3 Haiku 등이 효율적이며, 오픈소스 기반 LLM의 경우 Llama3(8B), Mixtral 8X7B, Llama 2 Chat(70B) 등이 상위권에 포진된 것을 확인할 수 있다. 한국어 처리에 대한 부분이 변수가 될 수 있으므로 모델 선정할 때 한국어 처리에 대한 부분도 확인해야 한다.

모델의 성능평가에는 벤치마킹 평가 뿐만 아니라 LLM 리더보드의 평가를 참고할 수 있다. 단일 LLM 리더보드에 의존하는 것보다는 복수의 LLM 리더보드 결과로 교차 검증하여 평가의 신뢰성을 높일 수 있다. 또한 개발자 커뮤니티의 의견이나 경험 결과를 추가로 고려하여 후보 모델을 선정한 후, 활용하고자 하는 실제 도메인에 적용하여 테스트하여 종합적으로 판단한다.

한국어 특화 LLM 리더보드 사이트는 다음과 같다.

- Open ko-LLM leaderboard (hosted by upstage, NIA)
 - 설명: AI-Hub의 한국어 데이터 셋을 활용하여 Upstage와 NIA가 공통으로 운영
 - URL: [https://huggingface.co/spaces/upstage/open-ko-ll](https://huggingface.co/spaces/upstage/open-ko-llm-leaderboard)

m-leaderboard

- LogicKor (hosted by instruct.kr)
 - 설명: 한국어 언어모델 다분야 사고력 벤치마크이며, instruct.kr팀이 운영
 - URL: <https://lk.instruct.kr/>
 - 구현소스 및 설명 참조: <https://github.com/instructkr/LogicKor>
- 금융 전문 LLM 리더보드 (hosted by 올거나이즈, <https://www.allganize.ai/>)
 - 설명: 금융 용어와 약어를 이해하고 복잡한 추론에 특화된 한국의 LLM 성능 평가
 - URL: <https://llm-leaderboard.org/>
- Horangi 한국어 LLM 리더보드 (hosted bywanda)
 - 설명: 자연어 이해 및 생성 관점에서 한국어 LLM 능력 평가
 - URL: <https://wandb.ai/wandb-korea/korean-llm-leaderboard/reportlist>

한국어에 특화되지 않은 LLM 리더보드 사이트는 다음과 같다.

- Open LLM Leaderboard (hosted by HuggingFaceH4)
 - 설명: 허깅페이스에서 운영하는 LLM 리더보드로 6가지 분야 성능 평가하여 순위 결정
 - URL: https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard
- LMSYS Chatbot Arena Leaderboard (hosted by LMSYS, <https://lmsys.org/>)
 - 설명: LMSYS에 의해 운영되며, 블라인드 테스트 방식으로 LLM 평가순위를 부여

·URL: <https://chat.lmsys.org/?leaderboard>

2.2.2 임베딩 모델

임베딩은 텍스트를 머신러닝 알고리즘에서 처리할 수 있는 숫자 벡터로 변환하는 NLP 기술이다. 벡터로 변환된 정보를 통해 단어/문장 간의 관련성을 계산할 수 있으며, 임베딩 된 정보가 벡터 저장소에 저장되어 텍스트의 의미와 관련된 처리를 수행한다.

일반적으로 OpenAI에서 제공하는 text-embedding-3 모델이 널리 사용되며, API 기반 임베딩 모델을 사용할 때마다 비용이 발생하므로 우수한 오픈소스 임베딩모델의 사용도 고려할 필요가 있으며, 오픈소스 임베딩 모델 중 baai/bge-m3 모델은 한국어 뿐만 아니라 다국어 임베딩 처리에 우수한 성능을 보여주며 성능 시험할 때 비교군으로 사용된 임베딩 모델들도 사용을 고려해볼 만하다[14].

2.2.3 벡터 데이터베이스

RAG 기반 LLM은 LLM에게 질문과 관련된 검색결과를 미리 알려줌으로써 환각을 줄이고 보다 정확한 대답을 생성할 수 있도록 하므로 목적지향 Chatbot의 목적에 부합하는 기술이며, 참고자료를 사전에 벡터DB 저장하고 질문이 들어오면 유사 데이터를 찾아서 LLM에 전달한다[3]. 많이 사용하는 벡터 데이터베이스는 크로마DB, Milvus, 엘라스틱서치, 파인콘등이 있다.

2.2.4 LLM 오케스트레이션 프레임워크

앞서 설명한대로, 임베딩된 데이터를 벡터DB에 저장하고 해당 내용을 LLM에 전달하는 전반적인

처리를 도와주는 프레임워크를 LLM 오케스트레이션 프레임워크라고 하는데, LLM서비스를 효율적으로 구현하기 위한 대표적인 오케스트레이션 프레임워크로는 Langchain, Semantic Kernel, LlamaIndex 등이 있다. 모델에 따라서 LLM 오케스트레이션 프레임워크를 지원하지 않거나 커스터마이징된 형태의 구현을 할 경우 LLM 오케스트레이션 프레임워크 없이 구현해야 되는 상황이 있으므로 평소 작동원리를 파악할 필요가 있다.

2.2.5 웹 프레임워크

Chatbot서비스를 위해 웹 프레임워크의 사용이 필수이며, Python 기반 웹 프레임워크는 Django, Flask, FastAPI가 있다. 많은 기능을 포함하는 Django가 많이 사용되고 있으나, 기능이 많고 복잡하기 때문에 학습 곡선이 높은 단점이 있다. Flask는 비교적 경량화 웹프레임워크로 마이크로 서비스 개발에 적합하며, FastAPI는 Go와 NodeJS와 함께 실시간 및 고성능 API구축에 특화되어 있고 특히 비동기 프로그래밍이 가능하다는 장점이 있어서 최근에 많이 사용되는 추세이다[15].

FastAPI 장점 중 하나는 Pydantic 라이브러리를 이용하여 OpenAPI 스키마(입력/출력)를 정확하게 생성한다는 점과 OpenAPI를 구현하는 도구로 Swagger를 사용하여 별도의 코딩없이 API명세서를 대몬이 기동될때 /docs 폴더에 접속하여 확인할 수 있으며, API Test도 지원한다. openAPI는 개방된 API의 의미인 Open API와는 다른 의미로 Restful API를 기 정의된 규칙에 맞게 API 스펙을 JSON이나 yaml로 표현하는 방식을 의미한다.

2.2.6 멀티프로세싱

안정적인 Chatbot서비스를 제공하기 위해서는

다중 접속자에 대한 답변 처리가 중요한데 Python에서는 socket과 threading 라이브러리를 이용하여 멀티 쓰레드 생성 요청과 TCP소켓서버를 연계한 API서버와 TCP소켓서버를 구성할 수 있다. 최근 대안기술로 Uvicorn과 FastAPI 통해 비동기 싱글 프로세스 API서버를 Gunicorn과 연동하여 멀티프로세스 기능을 구현할 수 있다[16].

2.2.7 동시성 제어

클라우드 기반 LLM API서비스 제공하는 회사들은 API 호출 제한(rate limit)을 두어 분당 호출할 수 있는 API 호출 횟수를 제한한다. 사용자의 API호출 횟수가 제한 값을 초과하면 에러를 반환하며 요청이 실패한다. 이러한 에러를 방지하기 위해 비동기화 기술 중 세마포어 기술을 활용하여 동시 요청 수를 제어할 수 있다.

세마포어는 멀티프로그래밍 환경에서 공유자원에 대한 접근을 제한하는 방법으로 Python에서는 `asyncio.Semaphore(n)` 함수를 활용하여 동시 수행하는 작업의 수를 조절할 수 있다[17].

2.2.8 데이터베이스 처리

FastAPI의 Pydantic과 잘 맞는 SQL 툴킷으로 SQLAlchemy가 널리 활용되고 있다. SQLAlchemy에서는 SQL문을 사용하여 데이터베이스 처리를 할 수도 있지만, ORM(Object Relational Mapper)개념을 활용하여 객체와 데이터베이스 간의 매핑을 간편하게 하여 처리 할 수 있다[15][18]. 이를 통해 테이블의 필드가 추가되거나 데이터베이스가 변경되어도 프로그램 코드를 거의 수정할 필요가 없어 개발과 유지보수가 편리하다. 파이썬에서 SQL문을 통해 데이터베이스를 처리하는 라이브러리로 `sqlite`, `pymysql`이 많이 사용된다. 참고로, JAVA진영은 파이썬의 ORM기술에 해당하는 기술이 JPA(Java Persistence API)라는 인터페이스가 있으며, 대표적인 JPA인터페이스의 구현체로 하이버네이트가 많이 사용된다.

2.3. RAG 기반 LLM 처리 프로세스

Patrick Lewis에 의해 제안된 RAG (Retrieval-Augmented Generation)는 외부지식베이스

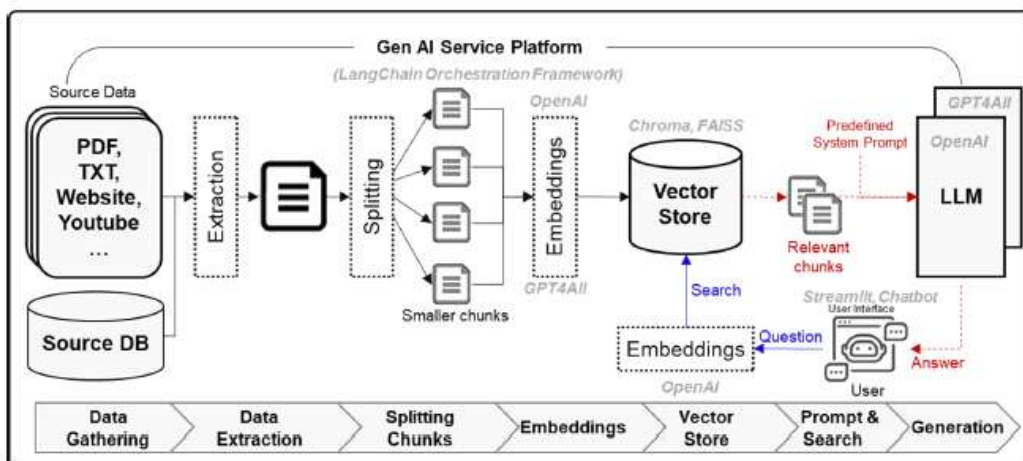


Fig 2. LLM과 RAG 기반 서비스 처리 프로세스[1]

에서 Input과 관련된 문서를 검색하여 실제 태스크를 수행하는 새로운 프레임워크였으며, 이와 관련된 연구가 활발히 진행되고 있다[4][19].

RAG 기반 LLM 처리의 기본적인 구조와 절차는 크게 두 가지 파트로 구분된다. 첫 번째는 질문 하기전에 데이터(웹, 파일, 텍스트 등)인 RAG 대상 문서를 전처리하고 벡터DB에 저장하는 것이다. 두번째는 LLM 추론 시 검색 결과와 질문을 LLM에 전달하여 LLM이 답변을 생성하는 것이다. 이때 RAG대상문서를 벡터DB에 저장하는 과정을 Chatbot 관리기능을 통해 진행할 수 있으며, LLM의 추론결과는 Chatbot을 통해 응답 처리할 수 있다. RAG 기반 LLM처리 절차에 대한 세부 프로세스는 Fig 2.과 같다.

2.4. LLM의 환각증상 개선연구

LLM의 매개변수에 관련된 지식이 부족하거나 최신정보로 업데이트되지 않은 경우, 정확한 추론이 어려운 한계가 있다[20]. 모델튜닝이나 모델재학습을 하지 않더라도 문맥학습(in-context learning)을 통해 개별 질의에 대해 정확한 답변을 유도할 수 있지만, 컨텍스트 정보가 많거나 계속 변경되는 경우에는 한 세션에 처리할 수 있는 토큰 수가 제한되고, 토큰 수에 따라 과금되는 방식이므로 이러한 문맥학습은 때로는 비효율적이다.

질문이 들어왔을때 RAG를 활용하여 컨텍스트 내용을 검색한 후 검색결과와 질문을 LLM에 함께 전달하여 답변을 요청하는 것이 보다 효율적이다[3]. 하지만 RAG 기반 LLM에 질문을 했을 때 Chatbot이 잘못된 답변을 하거나 답변을 제공하지 못하는 상황이 발생하면, 이에 대한 문제점을 파악하고 잘못된 지식을 바로 보정하는 것이 어려울 수 있다. 따라서 본 연구에서는 Chatbot 관리자에게 디버깅용 Chatbot을 제공하여 순수 LLM

답변, 벡터DB 검색결과, LLM&RAG 연계된 답변을 함께 볼수 있는 기능을 제안한다. 이를 통해 관리자가 어느 단계에서 잘못된 답변이 생성되었는지 파악하기 용이하다. 또한 사용자가 chatbot의 답변에 대해 오류를 신고하고, 관리자가 이를 쉽게 조치 할 수 있는 프로세스와 기능도 함께 제안한다.

3. 제안

현재, 다양한 연구들이 LLM의 한계를 극복하기 위해 청킹(Chunking), 한국어 임베딩, 프롬프트 엔지니어링, 그리고 RAG 검색 기능 개선을 위해 진행되고 있다. 본 연구는 이러한 처리 과정에서 좀 더 좋은 기술로 교체할때 프로그램 구조 변경을 최소화하는 유연하고 확장 가능한 아키텍처를 목표로 한다.

3.1. 구축 기본방향

Chatbot구축단계에서 Chatbot의 잘못된 답변을 튜닝하고 개선하고자 하는 연구는 활발한 반면, 시스템과 실시간 채팅이 이뤄지고 있는 상황에서 잘못된 답변에 대한 보고가 있을 경우에 대한 신속한 조치 방안에 대한 연구는 부족한 상황이다. 본 연구에서는 실시간 채팅 단계에서 신속히 답변에 대한 지식을 보정하고 반영할 수 있는 유사도 기반 Chatbot의 연계와 오류신고 절차 및 조치방안을 제시한다.

현재 많이 활용되고 있는 분류모델 기반의 인텐트 식별방식은 많은 구축 비용과 시간이 소요되는 문제가 있다. 이에 본 연구에서는 이러한 문제를 해결하기 위해 모델 학습 없이 유사문서 검색으로 인텐트를 식별하는 방안을 제안한다.

3.2. Q&A봇 구성

Q&A봇의 구성은 Chatbot API 서버, Chatbot 관리 API 서버, 시스템 관리 API 서버로 구성된다. Q&A봇의 각 종 API 호출은 업무지원 시스템 등의 응용 프로그램에서 호출할 수 있지만, 본 연구의 Q&A봇의 범위는 API 서버로 한정한다.

Q&A봇의 주요 개념으로 기본모드와 보정모드가 있으며, 프로젝트별로 관리된다. 기본모드에서 사용할 수 있는 기능은 프로젝트관리, RAG문서관리이며, 보정모드는 모든 기능을 사용한다.

기본모드는 RAG문서를 등록하고 추가 작업 없이 Q&A봇을 사용할 수 있으며, 보정모드는 Q&A봇이 답변을 못 했을 때 인텐트에 해당하는 질문-답변 쌍을 입력하여 답변을 보완할 수 있다.

Q&A봇의 시스템은 Fig 3.과 같이 구성하였다. 본 연구의 범위는 Q&A봇 엔진(솔루션)으로 한정하며, 응용프로그램은 포함하지 않지만, 연동에 대한 설명을 위해 일부 응용프로그램의 활용과 적용을 기술한다.

3.2.1 Chatbot API 서버

Chatbot API서버는 대화 추론과 대화 세션 관리를 담당한다. 대화 추론은 사용자의 질의 요청에 따라 시스템 관리 API 서버와 Chatbot관리 API 서버를 통해 등록된 정보를 기반으로 적절한 답변을 제공한다. 대화 세션 관리는 세션 ID 생성, 대화시작, 대화종료를 담당한다. 대화내역은 모두 대화 내역 DB에 기록된다. 유사 문서 검색 모듈에서는 기본적으로 TF-IDF벡터와 코사인 유사도를 사용하여 유사도를 평가하여 인텐트를 식별하며, 음절 일치율 방식의 옵션도 고려하고 있다. 유사도 산정방식은 프로젝트관리에서 선택한다.

3.2.2 시스템 관리 API 서버

시스템관리 API서버는 Chatbot 솔루션을 제공하는 회사의 시스템 관리자가 주로 사용한다. 고객사의 기본정보인 테넌트 정보를 관리하는 것이 이 서버의 주된 역할이다. 필요한 경우 사용자 정

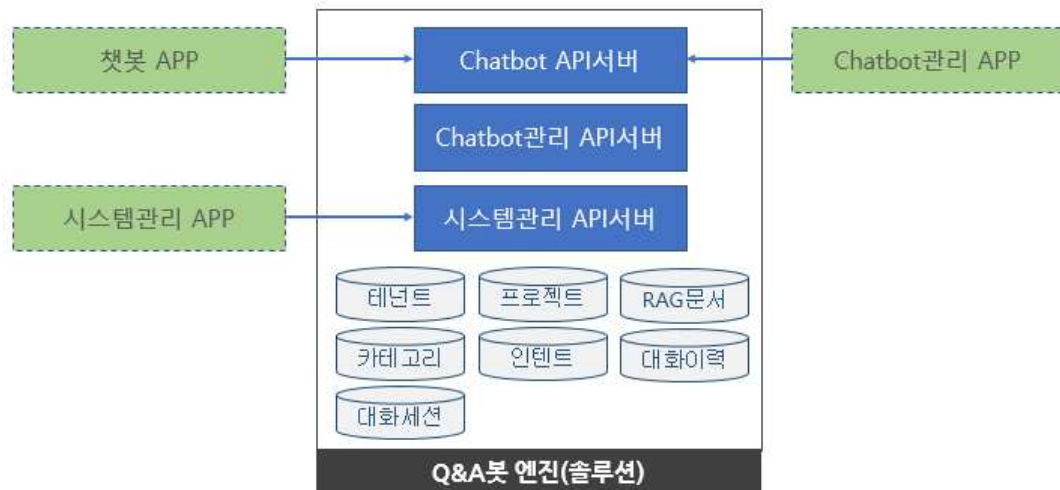


Fig 3. Q&A봇 엔진(솔루션)

보, 조직 정보를 관리할 수도 있다. 본 연구에서는 테넌트 정보만을 Q&A봇에서 관리하고, 사용자 정보와 조직 정보는 응용 프로그램에서 관리하고, 민감 정보에 해당하는 사용자 정보는 비식별 ID만을 각 API의 파라미터에 포함한다.

3.2.3 Chatbot관리 API서버

Chatbot이 동작하기 위한 각종 필요 정보를 관리한다.

- 프로젝트(봇)관리: 프로젝트이 전반적인 정보를 관리
- RAG문서 관리: 문서/웹/텍스트 형태의 문서를 등록하고 해당내용이 전처리를 거쳐 벡터로 등록
- (인텐트) 카테고리 관리: 인텐트와 관련된 카테고리 관리
- 인텐트 관리: 인텐트를 등록, 수정, 삭제할 수 있으며, 인텐트 내 복수의 질의를 구분자로 구분하여 등록

또한 Chatbot 관리 API서버는 다음과 같은 기능을 제공한다.

- 대화이력 및 오류신고관리: Chatbot 대화할 때 대화 이력이 자동으로 기록되며, 관리자는 오류 신고된 내용을 접수하고 인텐트를 매핑하여 오류 신고 조치를 완료

이외에도, Q&A봇의 동작모드에 따라 기능이 인텐트 기능을 사용할 수 있는데, 기본모드에서는 인텐트와 관련된 기능을 사용하지 않고 저장된 RAG문서를 통해 질문과 답변이 처리되며, 보정모드에서는 인텐트 관리를 통해 RAG 기반 LLM이 답변하지 못하는 내용에 대해 인텐트를 등록하여 처리할 수 있다.

기존의 분류모델 기반 Chatbot은 인텐트를 추가할 때 재학습이 필요했지만, 본 연구는 유사도 문

서 검색을 통해 학 습없이 인텐트를 식별하는 방법을 제안한다.

3.3. 유사문서 검색모듈

본 연구에서는 TF-IDF벡터와 코사인유사도를 활용한 유사도 산정 방식을 사용한다. 이 방식은 자연어처리에서 널리 사용되는 기술이다. 파라미터로 받은 질문과 인텐트에 속한 질의를 TF-IDF 원리에 따라 순차적으로 문자열을 벡터화하고 두 문자열의 관계를 코사인 유사도를 사용하여 산정한다. 필수 엔티티와 불용어 엔티티를 활용하여 발화 의도를 세분화한다.

예를 들어 “매트리스 교체시기 문의”, “정수기 교체시기 문의”라는 인텐트가 각 각 등록되었을 때 서로 다른 상품군으로 발화 의도가 잘못 인식 되는 일이 없도록 서로 필수 엔티티와 불용어 엔티티를 등록한다.

또한 유사도 임계치 조절하여 오인식이나 미인식 상황을 조절할 수 있다. 인텐트 구축 초반에는 낮은 임계치로 설정하여 더 많은 검색이 이뤄지도록 하고, 점차 임계치를 높여가며 적절한 임계치를 찾아가는 방식으로 진행할 수 있다.

엔티티를 변수처리하고 취환하는 방식으로 인텐트 내 시나리오 등록 갯수를 줄이는 방법도 있지만, 간단한 구조의 싱글톤 Q&A봇을 구현하는 것이 목적이므로 엔티티의 매개변수로의 활용은 고려하지 않는다.

3.4. 연동방안

LLM과 유사문서 검색 모듈의 연동은 조건이 충족되는 경우 유사문서 검색 모듈을 호출하는 방식으로 처리된다. 프로젝트(봇)가 보정모드이면서 인텐트에 해당 질의-답변 셋이 등록되어 있을

경우, 사용자 질문이 들어오면 Chatbot API서버는 유사문서 검색 모듈을 통해 사용자 질문과 유사한 질문이 존재하는지 확인하고 유사도 임계치를 넘는 가장 유사한 답변을 반환한다.

다음의 Fig4는 연동을 통한 Chatbot 동작 예이다. Fig3-(a)처럼 RAG 기반 LLM의 답변이 맞더라도 원하는 형태의 답변이 아닌 경우 관리자는 Fig3-(b)처럼 인텐트에 질문-답변 셋을 등록하고, 고객이 질의하면 인텐트의 답변이 Fig3-(c)처럼 바로 Chatbot화면에 노출될 수 있도록 설정할 수 있다.



Fig3. 업무 Q&A Chatbot 연동 예)

4. 아키텍처 및 구현

본 연구는 요구사항의 변동과 추가 기능 개발 요청이 있는 경우에도 비교적 확장이 용이한 애자일 방법론을 적용하여 LLM과 Chatbot을 결합한 Q&A봇 설계 및 구현을 하였다. Q&A봇의 구성은 3.2절에 기술한 것과 같이 Chatbot API서버, 시스템관리 API서버, Chatbot관리 API서버로 구성된다.

4.1 아키텍처

프로그램 호출 구조는 기본적으로 API 호출을

담당하는 APP과 비즈니스 로직을 수행하는 Biz로 구성된다. Pydantic 기반의 클래스는 OpenAPI의 스키마 개념을 따라 프로그램명 뒤에 schema(스키마)를 접미사로 붙였으며, SQLAlchemy 기반의 데이터베이스 레이아웃에 대한 클래스는 model이라는 명칭을 프로그램명 뒤에 접미사로 붙였다.

설계 및 구현에 활용된 기술 스택은 상황에 따라 변경 가능한 형태로 적용하였다.

- 다중 LLM: 기본 모델은 "gpt-3.5-turbo"로 구현하였으며, LLM 선정은 계약단계에서 결정되므로 테넌트관리에 부여된 LLM명코드를 항목으로 갖는다. 클라우드 기반 LLM API 개발 담당자와 설치형 LLM 서버 개발자의 개발 영역을 감안하여 클라우드형과 설치형 LLM을 위한 프로그램 파일로 분리 하였음.
- 다중 임베딩: OpenAI의 text-embedding-3모델로 개발하였으며, 오픈소스 baai/bge-m3모델을 추가할 예정
- 다중 벡터DB: 크로마DB로 구현하였으며, 향후 확장성을 위해, Milvus, 엘라스틱서치, 파인콘에 대한 코드를 부여하였다.

- LLM 오케스트레이션 플랫폼: Langchain으로 구현하며, 그외 LLM에 따라 적용
- 웹 프레임워크: FastAPI
- 트랜잭션방식 / 멀티프로세싱 : FastAPI와 Uvicorn의 동기식 메소드로 일괄 개발
향후 사안에 따라 Gunicorn연계, 비동기 메소드로 변경
- * Uvicorn: ASGI(Asynchronous Server Gateway Interface) Server, 비동기 단일 프로세스
- * Gunicorn : WSGI(Web Server Gateway\ Interface) HTTP Server, 단일 동기호출방식, Worker 설정을 통해 멀티쓰레드 구성
- 공유자원에 대한 접근제한 및 API호출 제한: 세마포어 등의 동시성제어 기술 활용
- 데이터베이스 처리 / 모델링: SQLAlchemy

4.2 응용프로그램 연동 구현

구현단계에서는 유연하고 확장 가능한 아키텍처를 기반으로 하여 다양한 이해관계자들의 참여를 통해 응용 프로그램을 개발하였다. 솔루션 개발자(API개발자, 모델서빙 개발자), 영업담당자, 서비스 기획자, 응용개발자, 경영진이 TF팀을 구성하여 응용 프로그램 개발에 참여하였다.

이 과정에서 프로그램 사용자의 편의성을 고려

하면서도 특정 기능이 다른 서비스에 영향을 주지 않는 것에 중점을 두었다.

Chatbot관리 API서버의 API를 응용프로그램에서 호출하여 구현하였으며, Chatbot관리 응용프로그램은 Fig. 4와 같이 AI봇 설정(프로젝트), RAG문서관리, 카테고리, 인텐트관리로 구성되었다. KMS 연동 등의 RAG 문서의 증가가 예상되어 RAG 문서용 카테고리를 추가할 예정이며, 인텐트용과 RAG문서용 카테고리로 분리하도록 설계 변경이 진행되고 있다. 카테고리 정보는 정보성이므로 RAG 문서 관리 기능에 대한 영향도는 크지 않다.

RAG 대상 문서를 등록할 때는 다수의 웹과 파일을 업로드하여 벡터DB에 저장할 수 있다. 시스템 부하를 줄이기 위해 RAG대상 문서 업로드 후에 적용버튼을 클릭하여 RAG 대상 문서가 벡터DB에 저장되도록 기획 및 구현하였다. 하지만, 이러한 방식이 사용자가 이해하기 어렵고 불편하다는 피드백을 반영하여 현재는 업로드와 동시에 RAG대상 문서가 벡터DB에 저장되도록 설계 변경을 진행하고 있다. 응용 프로그램에서 비동기 후처리로 호출하고, 벡터DB에 저장 여부에 대해 활성화/비활성화 상태에서 진행중 상태를 추가하여 3단계(비활성화, 진행중, 활성화) 상태로 관리한다.

No	AI 봇 이름	AI 봇 설명	프로젝트 타입	언어모델 타입	언어모델	상태
1	EICN 챗봇	EICN 챗봇	LLM 보정 모드	cloud	ChatGPT	완료

Fig 4. Chatbot관리 메뉴 구성

시스템에 부하를 조절하기 위해 한번에 처리 할 수 있는 건수를 제한하기로 협의하였다. 이러한 설계 변경을 통해 더욱 유연한 Chatbot관리가 가능해졌다.

그러나 벡터DB가 정상적인 검색을 수행하지 못할 때는 LLM은 관련 내용이 없다는 답변을 하거나 잘못된 답변을 하기도 한다.

Fig 6은 잘못된 Chatbot답변의 원인을 추적한 예이다.

시뮬 설정

RAG 문서 관리

카테고리

인텐트 관리

접기/펼기

검색

초기화

시뮬 선택

EICN 챗봇

전체 33 건

추가

No	제목	내용	완성화	문서 유형
1	EICN-회사소개	http://www.eicn.co.kr/bbs/board.php?bo_table=w1_c&wr_id=1	완성화	URL
2	AICC-STT	http://www.eicn.co.kr/bbs/board.php?bo_table=w2_1&wr_id=1	완성화	URL
3	AICC-TA	http://www.eicn.co.kr/bbs/board.php?bo_table=w2_4&wr_id=2	완성화	URL
4	클라우드-UB Contact	http://www.eicn.co.kr/bbs/board.php?bo_table=w3_2&wr_id=2	완성화	URL
5	컨택센터-IPCC	http://www.eicn.co.kr/bbs/board.php?bo_table=w4_1&wr_id=2	완성화	URL
6	컨택센터-보이는ARS	http://www.eicn.co.kr/bbs/board.php?bo_table=w4_2&wr_id=2	완성화	URL
7	2024 AI바우처 지원사업 공고	(공고문) 2024년 AI바우처 지원사업(5개분과) 과제 공고_제2024-0128호 (1).pdf	완성화	파일
8	2024년 블록체인 컨설팅 지원사업	2. [참고] 2024년 블록체인 컨설팅 지원사업 수요기업 모집공고.pdf	비완성화	파일

Fig 5. RAG문서관리

Chatbot의 주요기능은 벡터DB검색, LLM추론, 유사 문서 검색을 기반으로 한 인텐트 식별과 답변이다. 벡터DB 검색이 정확하게 이뤄지면 LLM이 검색결과를 문장형태로 생성한다.

본 연구에서는 잘못된 Chatbot 답변이 제공될 경우 쉽게 문제를 파악하기 위해 다음 세 가지를 각각 반환한다 : c)-① 순수LLM답변, ② RAG검색결과, ③ RAG 기반 LLM답변

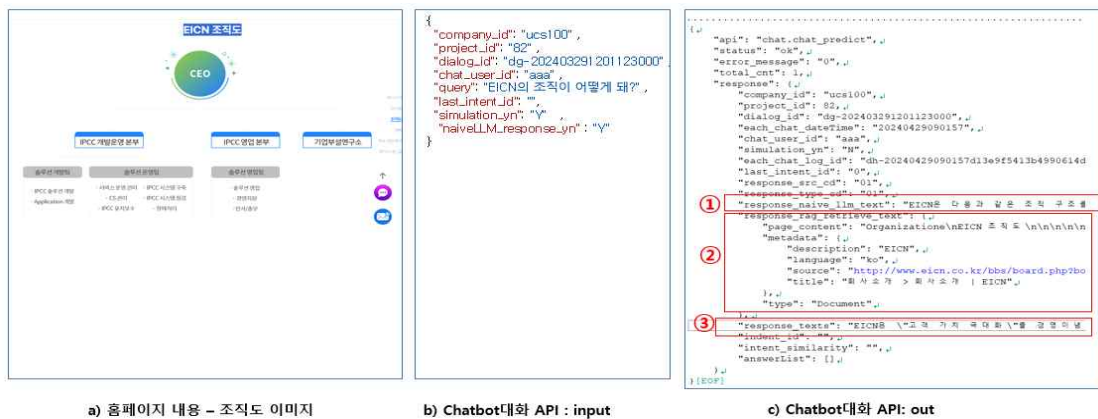


Fig 6. 잘못된 Chatbot답변 원인 추적 예)

예를 들어, 사용자가 EICN의 조직 구조에 대해 문의했으나, 순수LLM은 환각증상을 보이며 전혀 다른 답변을 하였으며, 벡터DB에서는 “EICN 조직” 정보 보다는 “EICN 조직” 문구 근처에 있는 “회사소개” 글을 더 많이 검색하였다. 이에 RAG 기반 LLM은 잘못된 검색 결과였던 “회사 소개”에 대한 설명을 바탕으로 잘못된 답변을 생성하였다. 이런 경우, 관리자는 Fig. 4의 예와 같이 올바른 답변을 인텐트에 등록하여 조치할 수 있다.

5. 결론

본 연구는 Chatbot구축과 관리를 위한 유연하고 확장 가능한 아키텍처를 제안하였다. 제안된 아키텍처는 사용된 기술 스택이 바뀌더라도 프로그램 구조에 대한 변경은 최소화하는 것을 목표로 설계하였으며, 구현 과정의 기술적 이슈와 필요 기술에 대해 논의하였다.

먼저, 실시간 채팅 단계에서 잘못된 답변에 대한 신속한 조치를 하기 위해 유사도 기반 Chatbot의 연계를 제안하고, 오류 답변 신고 절차와 조치 방안을 제시하였다.

특히 기존의 복잡한 구축 프로세스를 개선하기 위해 간단한 구조의 유사 문서 검색 기반 인텐트 관리를 제안하였다. 이는 관리자가 인텐트와 해당하는 질문-답변 셋을 등록함으로써 Chatbot이 적절한 답변을 할 수 있도록 하는 방식이다.

추가적으로 고려해야 할 부분은 기존 연구의 문제점으로 지적되었던 구분모델 기반 인텐트관리는 인텐트 수정할 때마다 모델학습이 요구되었으나, 최근 좋은 성능을 내는 생성형 LLM의 경우 모델 학습없이 프롬프트 러닝만으로도 수준 높은 분류가 가능하므로 LLM을 이용한 인텐트 식별도 고려할 수 있으나, 비용적인 측면을 고려할 필요가 있다. 또한 본 연구에서는 비교적 저비용의

간단한 방식의 유사 문서 탐색을 통해 인텐트관리를 구현하였지만, 벡터DB를 통한 질의응답 검색하는 방식도 고려해 볼만하다. 하지만, 벡터DB의 경우 전통 DBMS의 주된 기능을 지원하지 않고, 관리 난이도가 높아지므로 도입 시 이러한 사항을 고려해야 한다.

본 연구는 업무 Q&A 기반의 싱글턴 Q&A봇을 구현했지만, 앞으로 Q&A 기능을 포함한 대고객 Chatbot 시스템으로 기능을 확장 할 계획이다. 현재, 멀티턴 대고객용 콜봇의 설계와 기능 테스트를 진행하고 있으며, 대화엔진은 콜봇과 Chatbot이 공통으로 활용되 서로 상이한 로직은 분기하거나 추가될 수 있도록 진행하고 있다.

참고문헌

- [1] 신중민, 이재욱, 김경민, 이태민, 안성민, 박정배, 임희석 (2023), QA Pair Passage RAG 기반 LLM 한국어 챗봇 서비스, 한국정보과학회인어공학연구회 2023년도 제35회 한글 및 한국어 정보처리 학술대회, 2023 Oct. 12, 2023년, pp.683 - 689
- [2] 황금하, 권오욱, 이경순, 김영길 (2017), 목적지향 대화 시스템을 위한 챗봇 연구, 정보처리학회논문지. KIPS transactions on software and data engineering. 소프트웨어 및 데이터 공학, v.6 no.11, 2017년, pp.499 - 506, <https://doi.org/10.3745/KTSDE.2017.6.11.499>
- [3] 정천수 (2023), LLM 애플리케이션 아키텍처를 활용한 생성형 AI 서비스 구현: RAG모델과 LangChain 프레임워크 기반, 지능정보연구, v.29 no.4, pp.129 - 164, <https://doi.org/10.13088/jiis.2023.29.4.129>
- [4] Yunfan Gao and Yun Xiong and Xinyu Gao and Kangxiang Jia and Jinliu Pan and Yuxi Bi

- and Yi Dai and Jiawei Sun and Meng Wang and Haofen Wang (2024), “Retrieval-Augmented Generation for Large Language Models: A Survey” <https://doi.org/10.48550/arXiv.2312.10997>
- [5] Corrective Retrieval Augmented Generation (2024), <https://arxiv.org/abs/2401.15884>
- [6] Daniel Jurafsky & James H. Martin. 15. Chatbots & Dialogue Systems, <https://web.stanford.edu/~jurafsky/slp3>
- [7] Rishi Bommasani (2022), On the Opportunities and Risks of Foundation Model, <https://doi.org/10.48550/arXiv.2108.07258>
- [8] 최남규, 민병철, 조우리, 민경은, 정한결 (2022), 한국어 오픈도메인 대화시스템의 의 인화 연구: 사회적 변인에 따른 상대높임법 중심, ACK 2022 학술발표대회논문집, 29권 2 호
- [9] 류명균 (2022), 인텐트 분류 모델의 신뢰도 임계값이 인간-에이전트 소통에 미치는 영향 연구 : 태스크 지향 음성 에이전트를 중심으로, 서울대학교 대학원, 2022.8 <https://www.riss.kr/link?id=T16548927>
- [10] 유현조, 송영숙, 김민수, 윤기현, 정유남 (2021), 딥러닝 기반 한국어 개체명 인식의 평가와 오류 분석 연구, 한국언어학회, vol.46, no.3, pp.803 - 828, <http://dx.doi.org/10.18855/lisoko.2021.46.3.010>
- [11] 서민택, 나승훈, 임준호, 김태형, 류휘정, 장두성 (2023), 검색 증강 LLM을 통한 한국어 질 의응답, 한국정보과학회언어공학연구회, 제35 회 한글 및 한국어 정보처리 학술대회,
- [12] 강대욱, 나승훈, 김태형, 류휘정, 장두성 (2022), RAG를 이용한 한국어 오픈 도메인 질의 응답, 제 34회 한글 및 한국어 정보처 리 학술대회
- [13] LLM 비교: Quality, Speed, Price, <https://artificialanalysis.ai/models/mixtral-8x7b-instruct/providers>
- [14] Jianlv Chen and Shitao Xiao and Peitian Zhang and Kun Luo and Defu Lian and Zheng Liu, (2024), BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation, <https://doi.org/10.48550/arXiv.2402.03216>
- [15] FastAPI document, <https://fastapi.tiangolo.com/>
- [16] Unicorn과 uvicorn 연계 Document, <https://www.uvicorn.org/#running-with-gunicorn>
- [17] asyncio.Semaphore document, <https://docs.python.org/ko/3.10/library/asyncio-sync.html>
- [18] SQLAlchemy document, <https://www.sqlalchemy.org/>
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela (2021), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, <https://arxiv.org/abs/2005.11401v4>
- [20] 김산, 이근배, (2023), LLM을 활용한 오픈 도메인 대화 시스템의 유해성을 완화하는 데이터 증강 기법, 한국정보과학회언어공학연구회, 제35회 한글 및 한국어 정보처리 학술대회, pp.346 - 351, <https://scienceon.kisti.re.kr/srch/selectPORSrchArticle.do?>